

INFORME DE SEGUIMIENTO DE PRÁCTICAS HP SCDS

0. DATOS GENERALES

- **Título del proyecto:** *Cognitive Car Simulator* - Desarrollo de IA y Sistemas de Tráfico para Simulador de Conducción
- **Entidad:** HP-SCDS
- **Tutor de la entidad:** Luis Fuente Ibañez
- **Alumno:** Sergio Alves Martín
- **Fecha:** 7 de diciembre de 2025

1. OBJETIVOS DEL PROYECTO

El objetivo principal de mi participación en el proyecto es el desarrollo e implementación de la inteligencia artificial para vehículos, peatones y más entidades autónomas dentro del entorno de simulación CarSimulator, creada en Unreal Engine.

Las tareas específicas incluyen:

- **Control del Vehículo:** Implementar un controlador de IA (AIController) capaz de gestionar la dirección, aceleración y frenado del vehículo basándose en físicas reales (ChaosVehicleMovement).
- **Navegación:** Lograr que los vehículos sigan rutas aleatorias (BP_AI_Path) de manera precisa.
- **Sistemas de Tráfico:** Desarrollar la lógica de interacción entre los vehículos y los elementos del entorno, asegurando el respeto a las señales de tráfico.
- **Gestión de Versiones:** Mantener un control de versiones limpio y eficiente mediante Git/GitLab para el trabajo colaborativo.

2. ESTADO ACTUAL

Hasta la fecha, se han completado los siguientes hitos técnicos:

- **Reorganización y renombrado de carpetas y assets:**
 - El proyecto ha sido reordenado completamente, siempre usando PascalCase, en inglés y siempre siendo un nombre descriptivo y sin ambigüedad. Además, se ha añadido un prefijo que determina el tipo de asset, usando como referencia la propia guía que nos marca el foro oficial de Unreal.

- **Sistema de Semáforos Inteligentes (BP_TrafficLight):**
 - Creación de zonas de detección (SensorZone) mediante *Box Collisions*.
 - Implementación de comunicación bidireccional entre el semáforo y el coche. El semáforo detecta la presencia del vehículo (BeginOverlap) y envía órdenes directas al controlador.
 - **Solución de bloqueos (Deadlocks):** Se ha programado una lógica de "actualización de estado" (UpdateTrafficZone) que revisa los vehículos presentes en la zona cada vez que la luz cambia, garantizando que los coches reanuden la marcha inmediatamente al ponerse el semáforo en verde, incluso si ya estaban detenidos dentro de la zona.
- **Sistema de Frenado (En desarrollo):**
 - Implementación de la variable lógica ForceStop en el controlador para gestionar paradas obligatorias.
 - Inicio de la implementación de frenado progresivo basado en distancia, sustituyendo el frenado binario (0/1) por un cálculo gradual para mayor realismo.
- **Infraestructura y Git:**
 - Configuración correcta del archivo .gitignore para excluir archivos temporales de Unreal Engine y carpetas intermedias de Plugins (VictoryBPLibrary).
 - Limpieza del repositorio eliminando archivos binarios pesados (blobs > 100MB) de la base de datos de Visual Studio (.vs), permitiendo la correcta sincronización con GitLab.

3. PRÓXIMOS PASOS

Las tareas inmediatas para las próximas sesiones son:

1. **Finalizar el Frenado Progresivo:** Terminar de conectar la lógica matemática en el Event Tick del controlador para que la fuerza de frenado sea proporcional a la distancia del semáforo.
2. **Comunicación de Coordenadas:** Configurar el semáforo para que envíe la ubicación exacta de la línea de parada (StopTargetLocation) al vehículo al entrar en la zona de detección.
3. **Lógica del Ámbar:** Implementar el algoritmo de decisión para la luz ámbar (dilema de zona), donde el vehículo decidirá si frenar o acelerar dependiendo de si la distancia al semáforo permite una parada segura.
4. **Refactorización:** Limpieza de nodos de depuración (Print Strings) y optimización de Casts.

4. PLANIFICACIÓN:

- **(Sprint Actual)**
 - Tarea 1: Lógica de frenado suave (Map Range Clamped) en BP_AIController. [Estado: 80%]
 - Tarea 2: Envío de datos de ubicación desde BP_TrafficLight. [Estado: Pendiente]
 - Tarea 3: Algoritmo de decisión para semáforo Ámbar. [Estado: Pendiente]
 - Tarea 4: Pruebas de integración y ajuste de valores de física (fricción/fuerza de frenado). [Estado: Pendiente]
- **(Próximos Sprints)**
 - SPRINT 2: Vehículo Básico: creación de una IA con recorrido aleatorio, que obedezca las normas de circulación (semáforos).
 - SPRINT 3: Normas tráfico: implementación de cedas y stops a la IA de los vehículos.
 - SPRINT 4: Personalidades: implementación de personalidades a los vehículos (agresivo, inseguro, normal). Además de la implementación de aceras al mapa.
 - SPRINT 5: Pasos de cebrá y Peatón básico: añadir pasos de cebrá al mapa, y la implementación inicial de peatones en la ciudad del simulador.
 - SPRINT 6: Peatón normas: implementar a la IA del peatón el uso de pasos de cebrá y semáforos.
 - SPRINT 7: Interacción Vehículo - Peatón: ajustar IA del vehículo para que interactúe correctamente con los peatones (pasos de cebrá y semáforo).
 - SPRINT 8: Personalidades peatón: introducción de peatones normales que respetan las normas y otros con recorridos aleatorios (posibilidad de cruzar por la calzada)
 - SPRINT 9: Bicicleta Básica: implementar al simulador una IA de bicicletas, y que esta obedezca semáforos.
 - SPRINT 10: Normas bicicleta: ajuste de la bicicleta para que entienda correctamente las normas de circulación (stop, pasos de cebrá y cedas).
 - SPRINT 11: Pelotones y ubicación: implementar la posibilidad de pelotones y la ubicación de estos en el carril (centro o derecha).
 - SPRINT 12: Comportamiento bici y Compatibilidad: ajustar la bicicleta para que puedan ir en paralelo o en línea, además de ajustar la compatibilidad de estas con el resto de IAs.
 - SPRINT 13: IA Animales: implementar una IA con animales, ajustar para que estos puedan ir solos o en grupo y ajustar el resto de IAs para que interactúen correctamente con esta.
 - SPRINT 14: Assets y Diversidad: Implementación de nuevos assets a las IAs (diversidad/realismo).
 - SPRINT 15: Optimización: fine-tuning del programa, mejora del rendimiento, comportamiento, sonido, etc...)
 - SPRINT 16: Memoria TFG: Finalización de la elaboración de la memoria del TFG.